Universidade Estadual de Campinas
Instituto de Computação

Tarefa 2

Construção de Baralho

MC322 - Programação Orientada a Objetos

Professor: Marcelo da Silva Reis
PEDs: Caroline Nakazato, Giovanne Mariano, Gustavo Morais e Marcos Freire
PADs: Caio, Lia, Paulo e Vinicius

---

1. Descrição Geral

No Laboratório 1 foi implementado um sistema simples de batalha entre um herói e um inimigo, além das primeiras cartas que podem ser utilizadas pelo jogador. Entretanto, a implementação inicial contém diversas repetições de código. Por exemplo, tanto o herói quanto o inimigo possuem atributos e comportamentos semelhantes (como vida, defesa e recebimento de dano), e diferentes tipos de cartas compartilham características comuns, como nome e custo de energia.

Neste laboratório, o objetivo é **refatorar o projeto utilizando herança**, um dos conceitos centrais da programação orientada a objetos. A ideia é identificar comportamentos e atributos compartilhados entre classes e movê-los para **classes abstratas**, permitindo reduzir duplicação de código e melhorar a organização do sistema.

Para isso, deverão ser introduzidas duas novas classes abstratas:

- `Entidade` – representando qualquer personagem da batalha (como heróis e inimigos);
- `Carta` – representando o conceito geral de uma carta jogável.

As classes concretas existentes deverão passar a **herdar** dessas classes abstratas. Dessa forma, atributos e métodos comuns poderão ser definidos uma única vez nas superclasses e reutilizados pelas subclasses.

Além da refatoração estrutural, este laboratório também introduz elementos centrais do funcionamento de jogos do tipo *deckbuilder*. Em vez de utilizar um conjunto fixo de cartas disponíveis a qualquer momento, o jogador passa a utilizar um **baralho**, do qual cartas são compradas a cada turno.

Para implementar esse comportamento, deverão ser adicionadas as seguintes estruturas ao sistema:

- **Pilha de compra** – representa o baralho do jogador;
- **Mão do jogador** – cartas disponíveis para uso no turno atual;
- **Pilha de descarte** – cartas utilizadas ou descartadas durante a batalha.

A cada turno, o jogador deverá comprar cartas da pilha de compra para formar sua mão. As cartas utilizadas devem ser movidas para a pilha de descarte. Caso a pilha de compra se esgote, a pilha de descarte deverá ser embaralhada e transformada em uma nova pilha de compra.

Com essas modificações, o sistema passará a permitir uma **batalha completa baseada em baralho**, aproximando o funcionamento do jogo do modelo observado em *Slay the Spire*. O jogador deverá ser capaz de comprar cartas, escolher quais utilizar em cada turno e gerenciar sua energia até que o inimigo seja derrotado ou o herói perca toda a sua vida.

2. Objetivos

Ao final deste laboratório, o aluno deverá ser capaz de:

- Compreender e aplicar o conceito de **herança** em programação orientada a objetos;
- Utilizar **classes abstratas** para representar conceitos gerais de um sistema;
- Refatorar código existente para **reduzir repetição de atributos e métodos**;
- Estruturar uma hierarquia de classes utilizando superclasses e subclasses;

- Implementar o funcionamento básico de um **baralho de cartas**, incluindo pilha de compra, mão do jogador e pilha de descarte;
- Implementar o fluxo de compra e uso de cartas ao longo de múltiplos turnos de batalha;
- Gerenciar recursos do jogador, como **energia** para utilização de cartas;
- Expandir e reorganizar um sistema existente mantendo sua funcionalidade.

3. Atividades

Esta seção descreve a implementação principal do laboratório. O objetivo é **refatorar e expandir** o sistema desenvolvido no Laboratório 1. As classes apresentadas abaixo representam a estrutura mínima esperada, mas você é livre para adicionar novos atributos, métodos ou classes auxiliares.

As seguintes diretrizes gerais devem ser observadas:

- **Refatoração:** Sempre que possível, mova atributos e métodos comuns para as classes abstratas introduzidas neste laboratório, evitando duplicação de código;
- **Liberdade Criativa:** Você pode expandir o conjunto de cartas, criar novos inimigos ou adicionar classes auxiliares (por exemplo, para gerenciar o baralho). O tema do jogo continua sendo de livre escolha;
- **Autonomia Técnica:** Alguns detalhes de implementação são intencionalmente deixados em aberto. Espera-se que você:
  - Escolha tipos de variáveis adequados para cada atributo;
  - Defina parâmetros apropriados para cada método;
  - Tome decisões de design coerentes com os princípios de orientação a objetos.

Neste laboratório, o sistema deverá permitir que o jogador realize uma **batalha completa utilizando um baralho de cartas**. A cada turno, o jogador comprará cartas, escolherá quais utilizar, e as cartas usadas serão movidas para uma pilha de descarte.

3.1. Refatoração da Hierarquia de Classes

O primeiro passo deste laboratório é refatorar o código existente para introduzir duas classes abstratas.

3.1.1. Classe Abstrata `Entidade`

Representa qualquer personagem presente na batalha, como heróis e inimigos.

- **Atributos mínimos**
  - `nome` — Nome da entidade;
  - `vida` — Vida atual;
  - `escudo` — Escudo atual.
- **Métodos mínimos**
  - `receberDano` — Aplica dano considerando o escudo;
  - `ganharEscudo` — Adiciona escudo à entidade;
  - `estaVivo` — Retorna se a entidade ainda está viva.

As classes `Heroi` e `Inimigo` devem passar a **herdar** desta classe.

3.1.2. Classe `Heroi`

Representa o personagem controlado pelo jogador.

3.1.3. Classe `Inimigo`

Representa o oponente enfrentado pelo jogador.

- **Atributos sugeridos**
  - `dano` — Quantidade de dano causada ao atacar.
- **Métodos mínimos**
  - `atacar` — Realiza um ataque contra o herói.

3.1.4. Classe Abstrata `Carta`

Representa o conceito geral de uma carta jogável.

- **Atributos mínimos**
  - `nome` - Título da carta.
  - `descrição` - O que a carta faz.
  - `custo` - Custo em energia da carta.
- **Métodos mínimos**
  - `usar` — Executa o efeito da carta.

As classes concretas de cartas devem herdar desta classe.

3.1.5. Classes `CartaDano` e `CartaEscudo`

Essas classes devem representar cartas específicas do jogo.

- **CartaDano**
  - Deve causar dano a um inimigo quando utilizada.
- **CartaEscudo**
  - Deve conceder escudo ao herói quando utilizada.

Você pode criar outras cartas adicionais se desejar.

3.2. Sistema de Baralho

O jogador agora possui um **baralho de cartas**. Para isso, o sistema deve manter ao menos três coleções de cartas:

- **Pilha de compra** – cartas disponíveis para compra;
- **Mão do jogador** – cartas disponíveis para uso no turno atual;
- **Pilha de descarte** – cartas já utilizadas ou descartadas.

O fluxo esperado é o seguinte:

1. No início do turno, o jogador compra X cartas da pilha de compra;
2. As cartas compradas vão para a mão do jogador;
3. Quando uma carta é usada, ela vai para a pilha de descarte;
4. Se a pilha de compra acabar, as cartas na pilha de descarte devem ser embaralhadas e colocadas no topo da pilha de compra.

Você pode utilizar estruturas de dados da biblioteca padrão do Java, como `ArrayList` ou `Stack`, para implementar essas pilhas.

3.3. Sistema de Batalha

Atualize o sistema de batalha da classe `Main` para utilizar o novo sistema de baralho.
A cada turno:

- O herói recupera sua energia;
- O escudo do herói é zerado;
- O jogador compra cartas para formar sua mão;
- O jogador escolhe quais cartas utilizar;
- Cartas utilizadas vão para a pilha de descarte;
- Ao fim do turno do jogador, cartas não jogadas vão para a pilha de descarte;
- O inimigo realiza seu ataque.

O combate deve continuar até que:

- O herói seja derrotado; ou
- O inimigo seja derrotado.

3.4. Pontos de Atenção

Antes de finalizar o laboratório, verifique se:

- As classes `Heroi` e `Inimigo` herdam corretamente de `Entidade`;
- As classes de cartas herdam de `Carta`;
- A duplicação de código do laboratório anterior foi reduzida;
- O sistema de compra, uso e descarte de cartas funciona corretamente;
- A pilha de descarte é embaralhada quando a pilha de compra se esgota.

3.5. Desafio Extra (+0,5 ponto extra)

Figura 1: Um inimigo que pretende causar 6 de dano.

Em Slay the Spire, inimigos anunciam suas intenções no começo do turno no jogador, embora atuem somente ao término do turno. Por exemplo, na Figura 1, o inimigo pretende atacar o jogador causando 6 de dano. Este é um elemento importante do balanceamento do jogo, pois permite que o jogador monte sua estratégia de forma dinâmica, priorizando defesa se um inimigo pretende causar dano ou priorizando ataque caso contrário.

Como desafio extra neste laboratório, você deve implementar um mecanismo de anúncio de intenções de inimigos em combate, de modo que:

- O método de anúncio seja definido em uma superclasse `Inimigo`, da qual inimigos concretos devem herdar;
- Inimigos devem implementar o método de anúncio a depender do seu mecanismo de ataque;
- No começo de cada turno, junto com a informação dos inimigos, suas intenções também devem ser anunciadas.

Este desafio é inteiramente opcional, mas concluí-lo corretamente vale até +0,5 pontos extra na nota deste laboratório.

4. Avaliação

A nota do laboratório será composta pelos seguintes critérios:

1. **Modelagem e Refatoração da Hierarquia de Classes (3,0 pontos)**
   - Implementação correta das classes abstratas `Entidade` e `Carta`;
   - As classes `Heroi` e `Inimigo` devem herdar de `Entidade`;
   - As classes de cartas devem herdar de `Carta`;
   - Atributos e métodos comuns foram movidos para as superclasses, reduzindo repetição de código;
   - Uso adequado de encapsulamento (atributos privados e métodos públicos).
2. **Sistema de Baralho e Lógica de Batalha (3,0 pontos)**
   - Implementação da pilha de compra, mão do jogador e pilha de descarte;
   - Compra correta de cartas no início do turno;
   - Cartas utilizadas são movidas para a pilha de descarte;
   - A pilha de descarte é embaralhada quando a pilha de compra se esgota;
   - Aplicação correta de dano, escudo e custo de energia das cartas.
3. **Organização do Projeto e Qualidade do Código (2,0 pontos)**
   - Cada classe está em seu próprio arquivo `.java`;
   - Código legível, bem formatado e com nomes adequados;
   - Comentários quando necessários para trechos não triviais.
4. **Funcionamento Completo da Simulação (2,0 pontos)**
   - A batalha ocorre do início ao fim sem interrupções inesperadas;
   - O fluxo de turnos funciona corretamente (compra de cartas, uso de cartas e ataque do inimigo);
   - O combate termina corretamente quando o herói ou o inimigo é derrotado.
5. **Desafio extra (até +0,5 ponto extra)**
   - Conclusão correta do desafio proposto na Seção 3.5.
6. **Penalidade por Não Compilação**

   Caso o projeto:
   - Não compile utilizando Java 25; ou
   - Não execute corretamente no ambiente de correção;

   será aplicada uma penalidade de **-5,0 pontos** sobre a nota obtida.

   A nota final mínima é **0,0** (não haverá notas negativas).

5. Entrega

A submissão é realizada exclusivamente via repositório no **GitHub**. Para isso:

- Gere uma **release (tag)** com a identificação do laboratório, seguindo o formato: `tarefaX-RA1-RA2`;
- Gere uma release no GitHub a partir dessa tag;
- Envie o link da release no Google Classroom.

Caso optem por manter o repositório **privado**, adicionem os PEDs da disciplina como colaboradores:

- Caroline Nakazato – `@CarolineNakazato`
- Giovanne Mariano – `@giolucasd`

- Gustavo Morais – `@moraguma`
- Marcos Freire – `@marcfreir`

Lembrem-se que:

- O prazo de submissão é até o dia **19/03/2026 às 23h59**.
- Entregas atrasadas serão penalizadas em **25 % da nota obtida**.
- Os horários de laboratório e atendimento podem (e devem) ser utilizados para tirar dúvidas relacionadas tanto à submissão quanto ao desenvolvimento do laboratório.

5.1. Compilação e Execução

Durante a avaliação, os PEDs utilizarão os seguintes comandos, a partir da root do repositório, para compilar e executar o projeto:

```
javac -d bin $(find src -name "*.java")
java -cp bin App
```

Certifique-se de que existe uma classe chamada `App` contendo o método:

```
public static void main(String[] args)
```

Essa classe será utilizada como ponto de entrada para a execução do programa.
Dessa forma, a compilação e a execução ocorrerão corretamente no ambiente de avaliação.